High-Traffic Backend Simulation System

Project Architecture & Execution Plan

Objective	Simulate, observe, and optimize a backend system under increasing concurrent load
Stack	Spring Boot | PostgreSQL | Redis | Nginx | k6 | Grafana
Scale Target	~1,000 RPS | ~10,000 concurrent users
Setup	Two laptops — one server, one load-generator client
Methodology	Phased build: baseline → cache → horizontal scale → observe → experiment

01 Project Overview

This project builds a high-traffic backend simulation system designed to test how a CRUD-based application behaves under increasing concurrent load. The primary goal is not merely building functionality — it is to observe, measure, and understand how the system degrades under stress, where bottlenecks appear, and how optimizations like caching and horizontal scaling address them.

The system stores three categories of data — hot, cold, and static — and exposes a REST API with role-based access control. Load testing is performed progressively, comparing scenarios such as with and without caching, single versus multiple backend instances, and varying data access patterns.

Learning Outcome

Deep, hands-on understanding of request handling, database efficiency, cache performance, and system behavior under concurrent traffic — skills directly applicable to real-world engineering and system design interviews.

02 Frontend

The frontend is intentionally minimal. Its purpose is to serve as a functional UI client that exercises the backend APIs — not to demonstrate frontend engineering depth.

Technology Stack

- HTML, CSS, JavaScript — plain, no heavy framework
- Tailwind CSS for utility-first styling
- No build tooling required — keep it simple and fast to iterate

Pages

Authentication Page

- Login form and Sign-Up form
- Client-side validation using regex — email format, password strength, required fields
- Validation runs before the API call as a middleware layer in JS
- On success, JWT token received from backend is stored (localStorage or cookie)

Control Panel Page

The main page a user sees after login. Displays action buttons based on the user's role. Each button maps to a CRUD operation against the backend API.

Role	GET	POST	PUT	DELETE	PATCH
Admin	YES	YES	YES	YES	YES
First Class	YES	YES	YES	NO	NO
Second Class	YES	YES	NO	NO	NO

Buttons are conditionally rendered based on the role decoded from the JWT. No route should be accessible by manipulating the UI — enforcement is always server-side.

Middleware Layers (Frontend)

- **Auth:** Authentication middleware:
  - Checks for a valid JWT before allowing access to the control panel
  - Redirects to login if token is missing or expired
- **Authz:** Authorization middleware:
  - Reads the role claim from the JWT payload
  - Conditionally renders or disables buttons based on role
- **Validator:** Validator middleware:
  - Regex-based form validation on login/signup inputs
  - Runs before every API call to prevent malformed requests

03 Backend

Technology Stack

- Spring Boot — web framework, REST API layer
- Spring Security — authentication and authorization
- Hibernate ORM — object-relational mapping
- PostgreSQL — primary persistent data store
- Redis — hot data cache + JWT session store
- Nginx — load balancer and static file server

Architecture Layers

Request Lifecycle

- Client request hits Nginx on port 80
- Nginx routes to one of two Spring Boot instances (round-robin): port 8080 or port 8081
- Spring Security filter intercepts — validates JWT, checks role
- Controller receives request, delegates to Service layer
- Service layer applies business logic and calls Repository
- Repository (JPA/Hibernate) queries PostgreSQL
- For hot data reads, Service checks Redis before hitting Postgres
- Response travels back up the chain

Layer Breakdown

- Controllers — handle HTTP routing, request/response mapping, input validation via @Valid
- Service — business logic, cache-aside logic (check Redis → fallback to DB → populate Redis)
- Repository — JPA interfaces extending JpaRepository, zero boilerplate SQL for standard ops
- Entity — JPA-mapped POJOs representing DB tables

Spring Security & JWT

Spring Security manages both authentication (who are you?) and authorization (what can you do?). JWT is the token format. Because two instances run in parallel, JWT must be stateless from the application's perspective — tokens are validated cryptographically, not looked up in memory.

- On login: Spring Security validates credentials, generates JWT signed with a secret
- JWT payload carries: userId, username, role, expiry
- Every subsequent request includes the JWT in the Authorization header
- A JwtAuthFilter validates the token before each request reaches any controller
- Role-based access enforced via @PreAuthorize or HttpSecurity rules

Critical Design Note — Multi-Instance JWT

When running two Spring Boot instances, JWT cannot be stored in-memory (instance A won't know about tokens issued by instance B). Store active JWTs in Redis with a TTL matching token expiry. Both instances read from the same Redis. This also enables instant token revocation by deleting the Redis key.

Entities

- User — id, username, email, passwordHash (BCrypt), role (enum), createdAt
- Role — ADMIN, FIRST_CLASS, SECOND_CLASS (enum on User entity — no separate table needed)
- HotItem — id, itemKey (indexed), value (text/jsonb), updatedAt
- ColdItem — id, payload (large text), createdAt
- StaticAsset — id, filename, filePath, mimeType (metadata only — file lives on disk)

Data Layer Design

Hot Data

- Frequently read, infrequently written — product listings, config, user profiles
- Cache-aside pattern: check Redis first → on miss, query Postgres → store result in Redis with 60s TTL
- On write/update: delete the cache key (invalidation), never update cache directly
- Redis key format: hot:{id}

Cold Data

- Rarely accessed, potentially large payloads — audit logs, archived records
- Never cached — always reads directly from Postgres
- Demonstrates the latency penalty of uncached DB access under load

Static Data

- Images and files served directly by Nginx using alias directives
- Spring Boot is completely bypassed — no thread consumed for file serving
- This is one of the highest-impact optimizations in the experiment set

> **Performance Insight**
> Serving static files through Spring Boot under load consumes one Tomcat thread per file transfer. A 2MB image blocks a thread for the full transfer duration. Nginx serves the same file from OS-level sendfile() with no application thread consumed. Under load, this difference becomes dramatic.

Horizontal Scaling — Two Instances

- Two Spring Boot instances run on the same machine: ports 8080 and 8081
- Nginx round-robins requests between them
- Both instances share the same PostgreSQL and Redis — stateless application tier
- Tomcat thread pool default: 200 threads per instance — 400 total across both
- This is the first chokepoint to observe and tune under high load

04 Database

PostgreSQL — Primary Store

- Stores all persistent data: users, roles, hot items, cold items, static asset metadata
- HotItem.itemKey column should be indexed — it is the cache key and the most frequent lookup
- Connection pool managed by HikariCP (Spring Boot default) — start with max: 10 per instance
- Pool exhaustion is one of the first bottlenecks to appear under load — observe and tune
- Cold data table benefits from a createdAt index for range queries

Redis — Cache & Session Store

- Dual role: hot data cache AND JWT session store
- Hot data keys: hot:{id} with 60-second TTL
- JWT keys: jwt:{userId} with TTL matching token expiry (e.g. 1 hour)
- Both Spring Boot instances connect to the same Redis instance
- Redis is in-memory — target sub-millisecond reads, confirming cache value in load tests

Data Categories Summary

Category	Access Pattern	Storage	Caching Strategy
Hot Data	Frequent reads	PostgreSQL	Redis, 60s TTL, cache-aside
Cold Data	Rare reads	PostgreSQL	Not cached — DB only
Static Data	High volume	Disk (files)	Nginx serves directly (no app)

users

id	uuid	PK
username	varchar(50)	UK
email	varchar(255)	UK
password_hash	varchar(255)	
role	user_role	ENUM
is_active	boolean	
created_at	timestamptz	
updated_at	timestamptz	

hot_items

id	bigserial	PK
item_key	varchar(100)	UK + IDX
value	jsonb	
version	bigint	
created_at	timestamptz	
updated_at	timestamptz	

cold_items

id	bigserial	PK
payload	jsonb	
type	cold_item_type	ENUM
created_at	timestamptz	

static_assets

id	bigserial	PK
filename	varchar(255)	
file_path	varchar(512)	
mime_type	varchar(100)	
file_size	bigint	
uploaded_by	uuid	FK
created_at	timestamptz	

05 Load Testing Setup

Two-Machine Architecture

- Server laptop: hosts Spring Boot instances, PostgreSQL, Redis, Nginx
- Client laptop: runs k6 load scripts and Grafana dashboard
- Running load tests from a separate machine prevents CPU contention — the generator does not starve the server
- Machines communicate over local network (LAN) — keep on the same subnet for minimal network variance

k6 — Load Generator

- Writes test scripts in JavaScript targeting the server's IP and Nginx port 80
- Three test profile types to run across experiments:
  - Ramp-up: 10 → 50 → 100 → 200 RPS over ~3 minutes — finds degradation point gradually
  - Spike: sudden jump to max load — tests recovery and error handling
  - Soak: sustained moderate load for 10+ minutes — finds memory leaks and connection pool issues
- Thresholds to define as pass/fail criteria:
  - p95 latency < 500ms
  - Error rate < 1%

Grafana — Metrics Dashboard

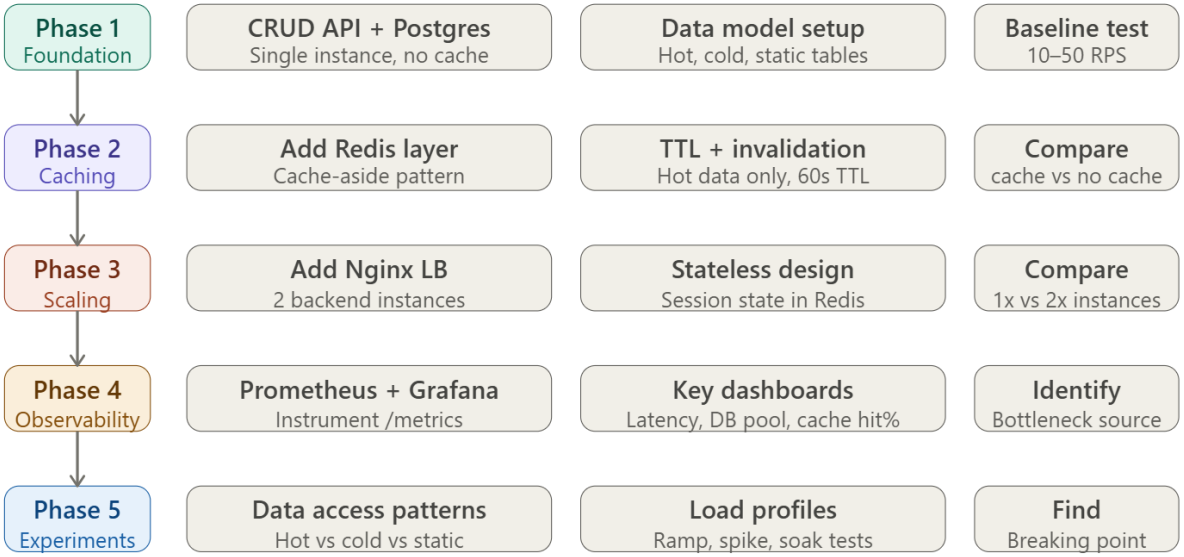
- Prometheus scrapes /actuator/prometheus from both Spring Boot instances
- Grafana reads from Prometheus and displays live dashboards
- Key panels to build:
  - p50 / p95 / p99 request latency over time
  - Requests per second (RPS) by endpoint
  - Redis cache hit ratio (hits / (hits + misses))
  - Active DB connection pool size vs max
  - Error rate by HTTP status code
  - CPU % per instance

Experiment Matrix

Exp.	Name	What It Measures
A	**Baseline**	No cache, single instance. Establish floor latency and error rate.
B	**Cache ON**	Add Redis. Measure cache hit ratio, DB query reduction, latency improvement vs Experiment A.
C	**2x Instances**	Add Nginx + second instance. Compare throughput ceiling and p95 vs single instance.
D	**Hot vs Cold Mix**	Serve a mix of hot and cold requests. Observe cache miss penalty and cold data p95 vs hot data p95.
E	**Static File Serve**	Compare Nginx-direct vs app-proxied static files. Measure throughput and latency delta under load.

06 Phased Execution Plan

The project is built and tested in five sequential phases. Each phase introduces one new concept, tests it in isolation, and establishes a new baseline before the next phase begins.



Experiment matrix

A: baseline (no cache, 1 instance) — establish floor latency and error rate

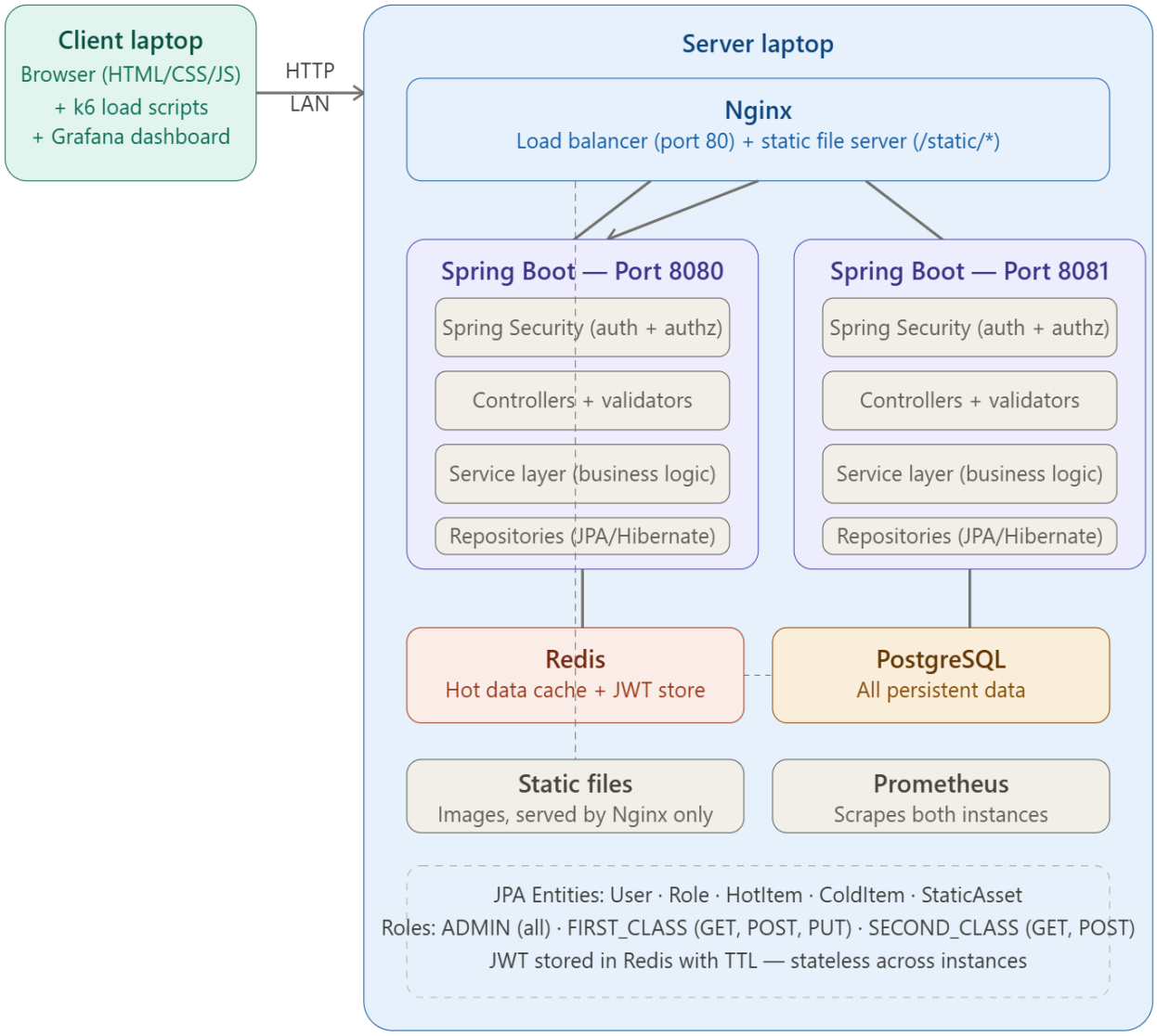
B: + Redis cache — measure cache hit ratio, DB query reduction, latency improvement

C: + 2 instances behind Nginx — compare throughput ceiling vs single instance

D: hot vs cold data mix — observe cache miss penalty, cold data p95 vs hot data p95

E: static file serving — Nginx direct vs app-proxied, measure throughput delta

Refined architecture



07 Topics to Study

The following areas were identified during planning as requiring additional learning before implementation. These are not blockers — they should be studied just-in-time as each phase begins.

k6 + Grafana Configuration

How to write k6 test scripts (VU model, stages, thresholds, checks). How to configure Prometheus as a Grafana data source. How to install and run Grafana on the client laptop. How to expose /actuator/prometheus from Spring Boot (add spring-boot-starter-actuator + micrometer-registry-prometheus dependencies).

Nginx as Load Balancer

How to write an nginx.conf that round-robins between two upstream servers. How to configure Nginx to serve static files directly via alias. How to reload Nginx config without downtime (nginx -s reload). Health check configuration for upstream servers.

Spring Boot Metrics & Actuator

Enabling the /actuator/prometheus endpoint. Configuring Micrometer for custom counters (cache hits, cache misses, cold data reads). Securing actuator endpoints so they are not exposed publicly. Tuning the HikariCP connection pool size and reading pool metrics.

08 Points to Reconsider

The following items were part of the original plan but are recommended for revision based on architecture review. None of these are critical changes — they are optimizations to the plan that will save time and reduce unnecessary complexity.

Drop the API Gateway

An API Gateway (Kong, AWS API Gateway) is designed for systems with multiple backend microservices — routing, rate limiting, auth delegation across services. This project has one backend service. Nginx already provides load balancing, static serving, and can handle basic rate limiting. Adding an API Gateway introduces configuration complexity, a new failure point, and a learning curve with zero benefit at this scale. Recommendation: remove API Gateway from scope entirely.

Never Serve Static Files Through Spring Boot

Serving images or videos through a Spring Boot controller means each file transfer holds a Tomcat thread for the full transfer duration. Under load, this exhausts the thread pool rapidly and degrades API performance. Nginx should serve all static files directly using an alias directive — Spring Boot is completely bypassed for /static/* paths. This is one of the highest-impact and easiest wins in the experiment set.

JWT Must Be Stateless Across Instances (or Stored in Redis)

If JWT validation state is kept in-memory (e.g. a blacklist HashMap in one instance), it breaks the moment Nginx routes a request to the other instance. Either use pure stateless JWT validation (signature + expiry only, no server-side lookup) or store active tokens in Redis so both instances share the same view. The Redis approach is recommended here because it also enables instant revocation, which is important for a security-conscious architecture.

Vertical Scaling is Not an Option Here (and That is Fine)

16GB RAM is a fixed constraint. Vertical scaling is off the table. This is actually the ideal constraint for this project — it forces horizontal scaling thinking, which is the more valuable skill. Two instances behind a load balancer on the same machine demonstrates the concept clearly even without separate physical servers.

09 Success Metrics

The project is complete when these outcomes have been observed, measured, and documented.

- Baseline breaking point identified (RPS where p95 > 500ms or error rate > 1%)
- Cache improvement quantified: p95 latency reduction (%) and DB query reduction (%) after adding Redis
- Horizontal scaling gain quantified: throughput ceiling (max sustainable RPS) with 1 vs 2 instances
- Cold vs hot data latency gap documented under load
- Static file serving improvement measured: Nginx-direct vs Spring-proxied throughput
- All five experiments run with reproducible k6 scripts and Grafana screenshots
- Final write-up: one table comparing all experiments, one paragraph explaining each finding

10 API Designing

- **Auth** — POST /api/auth/logout hits the backend intentionally. That's what deletes the JWT key from Redis, enabling actual revocation. A pure client-side logout (just clear localStorage) would leave the
- Redis key alive until TTL expires, which breaks your security model.

- **Users** — GET /api/users/{id} is open to all roles but the service layer enforces that non-admins can only fetch their own id. This is a common pattern — one endpoint, role-aware logic inside the service, not separate URLs per role.

- **Static assets** — Spring Boot only ever touches the metadata table. The actual file at filePath is served by Nginx directly via /static/*. So POST /api/static-assets assumes the file is already on disk — you'll need to decide later whether file upload goes through a separate Nginx-handled multipart upload or is pre-placed manually.

- **One thing to flag** — PATCH on hot/cold items is currently ADMIN-only. That might feel restrictive. If you want FIRST_CLASS to partially update too, easy to change. Just confirm before we move to design depth.

Auth (3 endpoints)
- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout

Users (7 endpoints)
- GET /api/users
- GET /api/users/{id}
- POST /api/users *(admin creates single user)*
- POST /api/users/bulk *(admin creates bulk users)*
- PUT /api/users/{id}
- PATCH /api/users/{id}/role
- DELETE /api/users/{id}

Hot Items (6 endpoints)
- GET /api/hot-items
- GET /api/hot-items/{id}
- POST /api/hot-items
- PUT /api/hot-items/{id}
- PATCH /api/hot-items/{id}
- DELETE /api/hot-items/{id}

Cold Items (6 endpoints)
- GET /api/cold-items
- GET /api/cold-items/{id}
- POST /api/cold-items
- PUT /api/cold-items/{id}
- PATCH /api/cold-items/{id}
- DELETE /api/cold-items/{id}

Static Assets (6 endpoints)
- GET /api/static-assets
- GET /api/static-assets/{id}
- POST /api/static-assets
- PUT /api/static-assets/{id}
- PATCH /api/static-assets/{id}
- DELETE /api/static-assets/{id}

Final Notes To Check

Cache Strategy — Lazy Loading (Cache-Aside)

- Cache-aside pattern chosen for read-heavy workload — check Redis first, fallback to DB on miss
- On cache miss, data is fetched from Postgres, written to Redis with 60s TTL, then served to client
- On write/update, cache key is invalidated immediately — never update cache directly

Thundering Herd Protection

- Problem: thousands of simultaneous cache misses all hitting Postgres at once — DB falls over
- Solution: distributed mutex lock using Redis SET NX PX (set if not exists with TTL)
- First request acquires lock → fetches from DB → writes to cache → releases lock
- All other requests on lock miss → spin-wait 50ms → re-read cache (now populated)
- Implementation: Redisson library or manual SET hot:{id}:lock NX PX 3000 in Spring Boot service layer
- Must be implemented before load testing — without it, Experiment B results are meaningless

Docker & Docker Compose

- All services containerized: both Spring Boot instances, PostgreSQL, Redis, Nginx
- Single docker compose up brings the entire stack live
- All containers on the same Docker network — Nginx reaches backends via container name (springboot-1:8080, springboot-2:8081), not localhost
- Memory limits set per container to prevent one service starving others — 512MB per Spring Boot instance, 256MB Redis, 256MB Postgres

Static File Serving

- Static files live on server laptop disk, bind-mounted as a Docker volume into the Nginx container
- Nginx serves files directly from disk — Spring Boot is never touched for static requests
- filePath column in static_assets table must match the exact bind-mount path — any divergence causes 404s and corrupts Experiment E results

Seed Data Strategy

- 10,000 users (split across FIRST_CLASS and SECOND_CLASS), 10,000 hot items, 10,000 cold items, 10,000 static files
- Static files generated as random binaries of varying sizes (5KB, 50KB, 500KB) — no real images needed, realistic file sizes are what matter for throughput measurement
- Bulk user seeding via POST /api/users/bulk endpoint using JPA saveAll() for batched inserts

Frontend Architecture

- Plain HTML/CSS/JS served directly by Nginx — no framework, no build tooling
- This is SSR-lite: Nginx serves the static HTML shell, JS makes API calls to backend
- No React or JS bundle overhead — correct choice for a load-testing client UI, not a product

Observability & Metrics

- Prometheus scrapes /actuator/prometheus from both Spring Boot instances
- Grafana reads from Prometheus, live dashboards during load tests
- Metrics tracked: cache hit ratio, error rate, RPS, throughput, DB connection pool usage, p95 latency, **p99 latency**
- p99 is critical — under thundering herd, p95 can look healthy while p99 is 10x worse; outliers are where the real behavior lives

**********End of Document**********